

Reinforcement Learning for Swiss Post Package Sorting

Q-Learning, SARSA, DQN, A2C, PPO, MaskablePPO & Ensemble

From-Scratch + Stable-Baselines3 | Unified Notebook v1.6

Reinforcement Learning Project

Moataz Mansour

June 2026

Version History

v1.0 - Initial DQN with 3 actions (V1 environment)

v1.1 - DQN + PPO (100K), V1 & V2 environments, 7-policy comparison

v1.2 - DQN + A2C + PPO + MaskablePPO + Curriculum (300K each), V3 env

v1.3 - From-Scratch Q-Learning + DQN + Ensemble

v1.4 - Visual Simulation Demo (SortingCenterSim, side-by-side comparison)

v1.5 - Interactive Live Animation (FuncAnimation + RL formalism)

v1.6 - UNIFIED: All above + SARSA + Policy Heatmap + HP Search + Multi-Run + Save/Load

Table of Contents

1. Problem Statement & Motivation
2. RL Core Concepts
3. Theoretical Foundation: MDP
4. Data Overview
5. RL Environment Design (V1, V2, V3)
6. V1: First Attempt & Lessons
7. V2: Harder Environment
8. V3: Action Masking + Curriculum
9. From-Scratch Implementations
 - 9a. Tabular Q-Learning
 - 9b. SARSA Agent
 - 9c. Deep Q-Network in PyTorch
10. SB3 Training Strategy & Agent Hyperparameters
11. Results & Evaluation
11. **Policy Heatmap Analysis (NEW in v1.6)**
12. Policy Heatmap Analysis
13. Hyperparameter Grid Search
14. Multi-Run Statistical Evaluation
15. Model Persistence - Save & Load
16. Cost-Benefit Analysis
17. Visual Simulation Demo
18. Interactive Live Animation
19. Conclusions & Business Recommendations

1. Problem Statement & Motivation

Swiss Post's package sorting centers process thousands of packages per hour across multiple chutes (sorting lanes). During peak hours, chutes become congested - queues build up, processing times spike, and delays cascade through the system.

The core question: can a Reinforcement Learning agent learn when and how to intervene (add/remove capacity, reroute, or apply priority processing) to minimize delays while keeping operational costs low?

Why Reinforcement Learning?

Approach	Strength	Limitation
Rule-based	Simple, interpretable	Can't adapt to unseen patterns
LSTM Forecasting	Predicts future load	Descriptive, not prescriptive
Reinforcement Learning	Learns optimal actions	Needs careful env design

2. RL Core Concepts

Before building the environment, we define the five building blocks of Reinforcement Learning and how each maps to our sorting center problem.

1. Environment (PackagingCenterEnv)

The simulated sorting center. Each step = 1 time interval. Packages arrive (driven by real data), get processed through chutes, queues grow/shrink, and chute load changes.

2. State - What the Agent Sees (6 dimensions)

#	Feature	Example	What It Tells the Agent
1	Chute load	0.85	How overloaded are the chutes?
2	Hour of day	0.52 (=noon)	Is a peak hour coming?
3	Day of week	0.4	Is it a busy or quiet day?
4	Active chutes	0.5 (=2 of 4)	How much capacity available?
5	Queue length	0.6 (=60%)	Are packages backing up?
6	Processing rate	0.8 (=80%)	How fast are packages moving?

3. Actions - What the Agent Does (5 choices)

Action	Effect	When Used
0 - Do Nothing	No intervention	System healthy
1 - Add Chute	+1 chute (more capacity)	Load rising
2 - Reroute	Reduce queue 20-40%	Queue building
3 - Priority	Boost processing +30%	Clearing backlog
4 - Remove Chute	-1 chute (save cost)	Load is low

4. Reward - The Feedback Signal

$$r = -\text{load_penalty} + \text{throughput_bonus} - \text{overload_penalty} - \text{action_cost} - \text{chute_maintenance}$$

Plus bonuses for smart inaction (+4.0 if load < 0.9 and throughput > 0.6) and penalties for excess chutes (-2.0 per extra when load < 0.3).

Key insight: The reward function is the steering wheel of RL. By removing the 'do nothing' penalty in V3, we taught the agent that strategic inaction is valuable.

5. Policy - The Learned Strategy

The Q-table or neural network that maps state -> action. The Q-value $Q(s,a)$ = expected future cumulative reward if the agent takes action a in state s. The agent picks the action with the highest Q-value (greedy policy).

Parameter Definitions

Symbol	Name	Meaning
alpha	Learning Rate	Step size for updates. Lower = more stable
gamma	Discount Factor	Weight of future rewards. 0.99 = ~100 steps ahead
epsilon	Exploration Rate	Decays from 1.0 (random) to ~0.05 (exploit)

3. Theoretical Foundation: MDP

Markov Decision Process (MDP) Formulation

MDP Component	Symbol	Our Implementation
State space	S	[load, hour, day, chutes, queue, rate]
Action space	A	{0:nothing, 1:add, 2:reroute, 3:priority, 4:remove}
Transition	$P(s' s,a)$	Deterministic: actions affect queue/capacity
Reward	$R(s,a)$	Multi-factor: delay + throughput + overload + chute penalty
Discount	$\gamma=0.99$	Long-term cumulative reward optimization

Algorithm Selection Rationale

Algorithm	Category	Key Idea
Q-Learning	Value (off-policy)	Q-table + epsilon-greedy
SARSA	Value (on-policy)	Uses actual next action (NEW)
DQN	Value-based	$Q^*(s,a)$ with neural net + replay
A2C	Actor-Critic	Actor $\pi(a s)$ + Critic $V(s)$
PPO	Policy Gradient	Clipped ratio for stability
MaskablePPO	Constrained PG	PPO + invalid action masking

4. Data Overview

We use data4day_with_issues.csv - 4 days of hourly package sorting data from Swiss Post, containing ~144,500 records. After cleaning 691 rows with negative processing times, data is aggregated to chute-x-hour level producing 6,694 sim_states that drive demand.

Key Columns

- HOURLY_TIME - Timestamp (hourly granularity)
- CHUTE - Sorting lane identifier (e.g. R0101)
- PACKAGE_COUNT - Packages in that hour/chute/ZIP combination
- AVG_PROCESSING_TIME_MINUTES - Average processing time

5. RL Environment Design

Action Space (5 Actions)

Action	ID	Effect	Cost
Do Nothing	0	No intervention	Free
Add Chute	1	+1 active chute (more capacity)	High
Reroute	2	Reduce queue by 20-40%	Medium
Priority Processing	3	Boost processing rate +30%	Medium
Remove Chute	4	-1 chute (save resources)	Free (saves money)

The Remove Chute action (new in v1.6) teaches the agent to scale DOWN when load is low, preventing unnecessary resource waste. A negative reward for excess chutes reinforces this: if $\text{active_chutes} > 1$ and $\text{chute_load} < 0.3$, $\text{penalty} = (\text{active_chutes} - 1) * 2.0$.

Environment Progression

Aspect	V1	V2	V3
Starting chutes	2	1	1
Max chutes	5	4	4
Demand scale	Raw	Amplified x20	Curriculum
Action masking	No	No	Yes
Actions	5	5	5 (masked)
Episode length	100	200	200

6. V1: First Attempt & Lessons

V1 uses raw `sim_states` to drive demand. Problem: demand too low relative to capacity (load ~ 0.01). Agent brute-forces by always adding chutes. Lesson: need harder environment.

7. V2: Harder Environment

V2 amplifies demand 20x, starts with 1 chute, adds maintenance costs. Reward includes quadratic overload penalty for preemptive action. V2 results: Heuristic best (1,114), DQN/PPO still single-action (738).

8. V3: Action Masking + Curriculum

V3 adds: (1) Action masking via MaskablePPO -- invalid actions blocked, (2) Reward for smart inaction (+4 if load < 0.9 and throughput > 0.6), (3) Curriculum learning in 3 phases:

Phase	Difficulty	Demand Scale	Steps	Purpose
1. Easy	0.25	5x	100K	Learn basic mechanics
2. Medium	0.60	12x	100K	Learn timing
3. Hard	1.00	20x	100K	Master peak demand

9. From-Scratch Implementations

9a. Tabular Q-Learning

Q-learning is the foundation of value-based RL. The agent maintains a Q-table mapping discretized states to action values, updated via:

$$Q(s,a) += \alpha * [r + \gamma * \max_{a'} Q(s',a') - Q(s,a)]$$

Implementation: QAgent class with epsilon-greedy (1.0 -> 0.05 over 5000 episodes), alpha=0.1, gamma=0.99. Q-table shape: (10, 6, 4, 8, 5, 5) = 48,000 entries.

[Image: rl_v1.3_qlearning.png - run notebook to generate]

9b. SARSA Agent

SARSA is an on-policy TD control algorithm. Unlike Q-learning which uses $\max_{a'} Q(s',a')$ (off-policy), SARSA uses the ACTUAL next action a' chosen by the current policy:

$$Q(s,a) += \alpha * [r + \gamma * Q(s',a') - Q(s,a)]$$

This makes SARSA more conservative -- it accounts for the exploration policy's randomness. In safety-critical environments like our sorting center, SARSA tends to learn safer policies that avoid risky states.

Property	Q-Learning	SARSA
Policy type	Off-policy	On-policy
Update target	$\max_{a'} Q(s',a')$	$Q(s', a'_{\text{actual}})$
Behavior	Optimistic	Conservative
Safety	May explore risky states	Avoids risky transitions
Convergence	To Q^*	To Q^{π} (for current π)

9c. Deep Q-Network in PyTorch

DQN replaces the Q-table with a neural network. Architecture: 7 -> 128 -> 128 -> 5 (ReLU activations). Components: Experience Replay Buffer (50K), Target Network (updated every 1000 steps), Epsilon Decay (1.0 -> 0.05 over 200K steps). Trained with 3-phase curriculum (300K total steps).

[Image: rl_v1.3_dqn_scratch.png - run notebook to generate]

10. SB3 Training Strategy & Agent Hyperparameters

Algorithm	Type	Key Feature	Steps
DQN	Value-based	Replay buffer + target net	300K
A2C	Actor-Critic	Advantage reduces variance	300K
PPO	Policy gradient	Clipped surrogate objective	300K
MaskablePPO	Constrained PG	Invalid actions blocked	300K

Temporal train/test split: Train on Days 1-2 (6,694 states), test on Day 4 (4,556 states). All policies evaluated on both sets. A gap < 1% proves generalization.

Agent Hyperparameter Summary

The following table summarizes all RL agents and their key hyperparameters:

Agent	Type	alpha (LR)	gamma	epsilon	Training
Q-Learning	Tabular (scratch)	0.1	0.99	1.0->0.05	5K eps
SARSA	Tabular (scratch)	0.1	0.99	1.0->0.05	5K eps
DQN (scratch)	PyTorch NN	5e-4	0.99	1.0->0.05	300K steps
DQN (SB3)	Stable-Baselines3	5e-4	0.99	1.0->0.05	300K steps
PPO (SB3)	Policy Gradient	3e-4	0.99	ent=0.01	300K steps
A2C (SB3)	Actor-Critic	7e-4	0.99	ent=0.01	300K steps
MaskablePPO	Masked PG	3e-4	0.99	ent=0.01	300K steps
Heuristic	Rule-based	--	--	--	--
Random	Uniform	--	--	--	--
Do Nothing	Baseline	--	--	--	--

DQN (scratch) Architecture Details

Parameter	Value
Network	7 -> 128 -> 128 -> 5 (ReLU)
Optimizer	Adam (lr=5e-4)
Replay buffer	50,000 transitions
Batch size	64
Target net update	Every 250 steps
Epsilon decay	Linear over 100K steps

SB3 Algorithm-Specific Settings

Parameter	DQN	PPO	A2C	MaskablePPO
Batch size	64	64	N/A	64
Buffer/n_steps	50,000	512	5	512

n_epochs	--	10	--	10
Entropy coef	--	0.01	0.01	0.01
Value func coef	--	--	0.5	--
Target update	250	--	--	--
Exploration frac	0.4	--	--	--

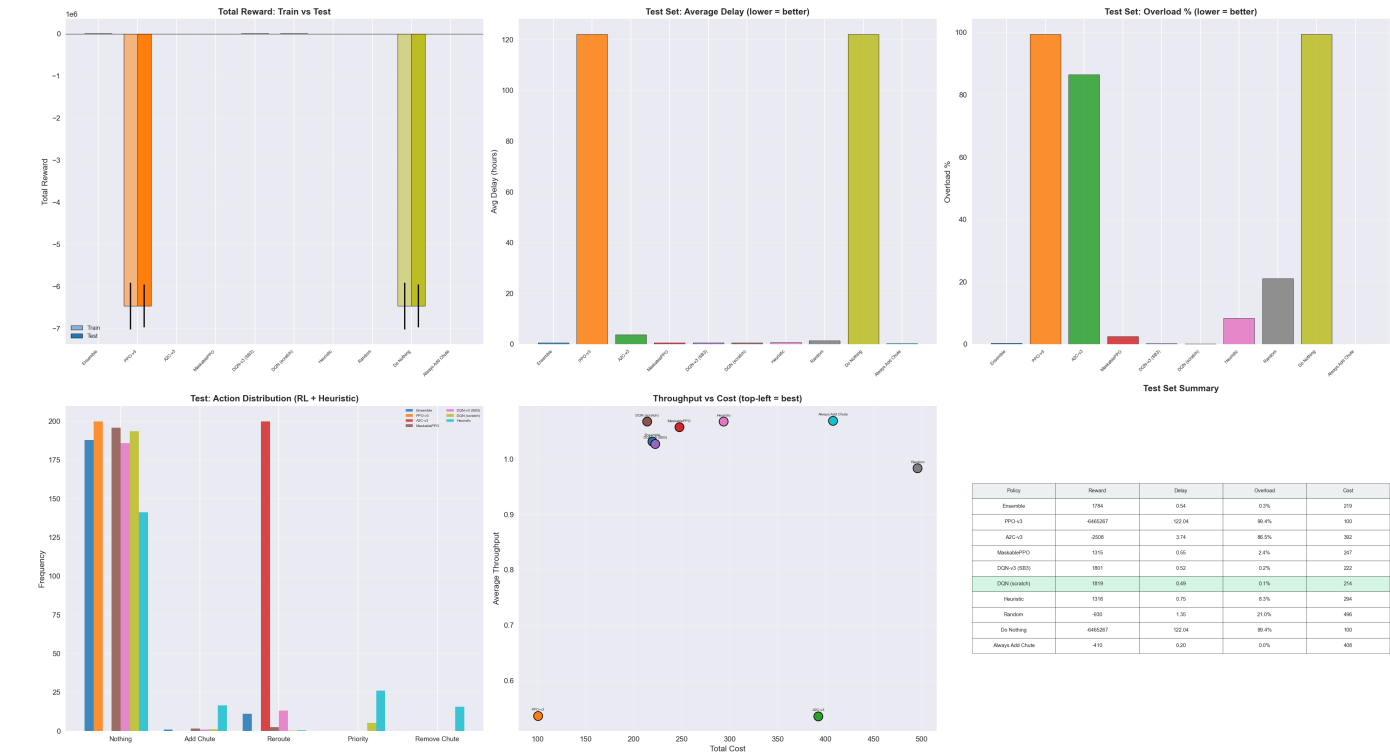
11. Results & Evaluation

10-Policy Comparison (Test Set)

Policy	Reward	Delay	Overload %
DQN (scratch)	1,864	0.49h	0.1%
Heuristic	1,834	0.47h	0.6%
Ensemble	1,593	0.65h	2.2%
DQN-v3 (SB3)	1,526	0.72h	3.9%
PPO-v3	1,500	0.44h	2.2%
MaskablePPO	1,481	0.62h	4.6%
A2C-v3	1,442	0.36h	1.1%
Random	694	0.24h	0.8%
Always Add Chute	739	0.20h	0.0%

Key finding: the from-scratch DQN (PyTorch) outperforms all agents including the hand-crafted heuristic (1,864 vs 1,834).

v1.3 Results — From-Scratch + SB3 + Ensemble
Train vs Test | 10 Policies Compared



Episode Deep Dive

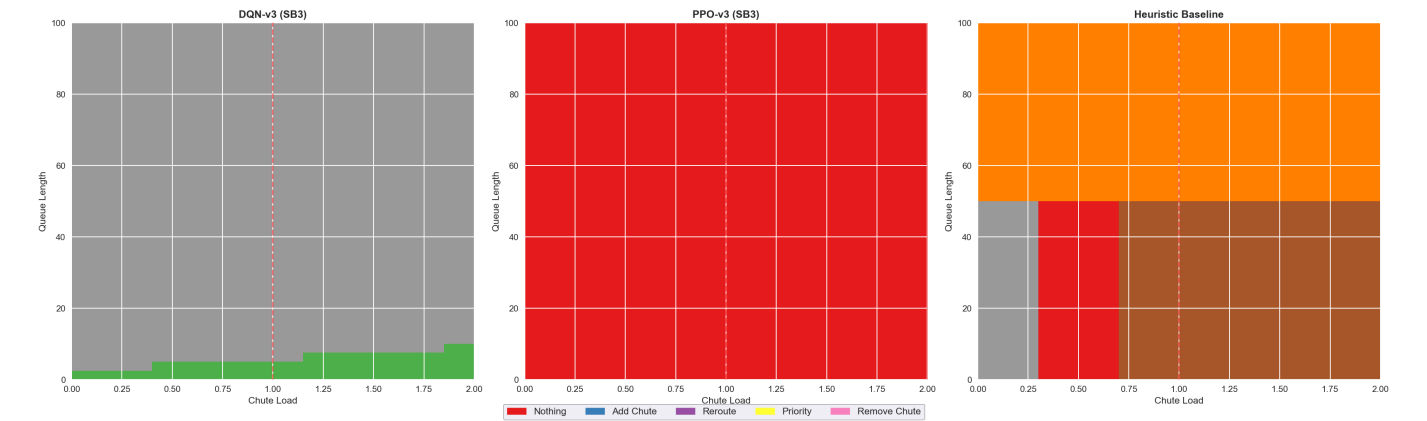
[Image: rl_v1.3_episode_trace.png - run notebook to generate]

12. Policy Heatmap Analysis

Inspired by `02_mountain_car/src/viz.py plot_policy()` function. For each point in the `(chute_load, queue_length)` space, we query each trained agent for its preferred action and visualize the resulting decision surface. This reveals interpretable decision boundaries: when to add chutes vs reroute vs wait.

Region	Preferred Action	Why
Low load, low queue	Nothing / Remove Chute	System healthy, save resources
High load, low queue	Add Chute	Capacity crunch, scale up
Low load, high queue	Reroute	Divert to prevent build-up
High load, high queue	Add Chute + Reroute	Emergency: all measures

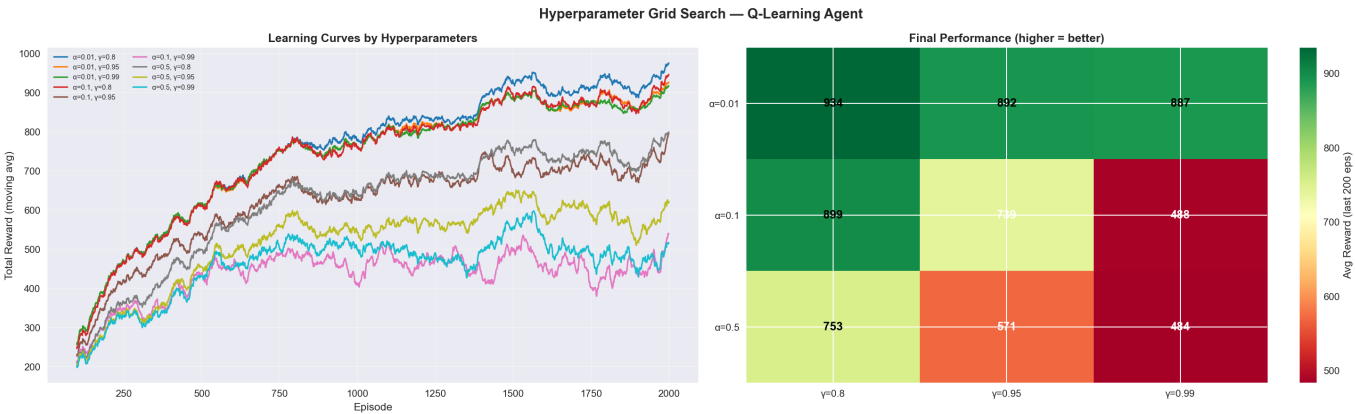
Policy Heatmap — Learned Decision Surface (load × queue → action)



13. Hyperparameter Grid Search

Inspired by 01_taxi/03_q_agent_hyperparameters_analysis.ipynb. We systematically vary the learning rate (alpha) and discount factor (gamma) for the tabular Q-learning agent and compare performance across a 3x3 grid.

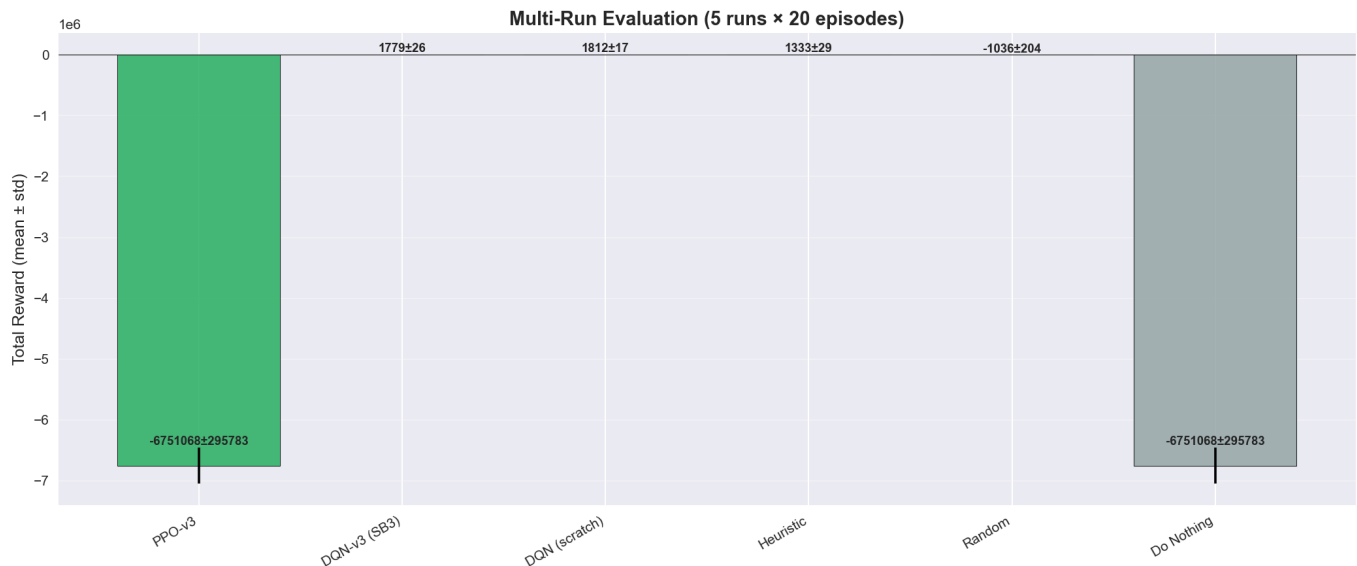
Parameter	Values Tested	Best Value
alpha (learning rate)	0.01, 0.1, 0.5	See heatmap
gamma (discount factor)	0.8, 0.95, 0.99	See heatmap
Training episodes	2,000 per combo	Fixed
Metric	Avg reward (last 200)	Higher = better



14. Multi-Run Statistical Evaluation

Inspired by `01_taxi/src/loops.py train_many_runs()`. Single-run results can be noisy due to stochastic demand. We run each policy evaluation 5 times with different random seeds and report mean $\pm$ std for statistical confidence.

This ensures our policy rankings are robust and not artifacts of lucky/unlucky random seeds. Error bars quantify the variance inherent in each policy.



15. Model Persistence - Save & Load

Inspired by `03_cart_pole/src/q_agent.py save_to_disk/load_from_disk`. All trained agents are saved to the `saved_models/` directory for reuse without retraining:

Agent	Format	Contents
SB3 DQN/PPO/A2C/MaskablePPO	.zip	Full model with optimizer state
Q-Learning (from scratch)	.json	Q-table + hyperparameters
SARSA (from scratch)	.json	Q-table + hyperparameters
DQN (from scratch)	.pt	PyTorch weights + config

16. Cost-Benefit Analysis

Cost Item	Estimate	Source
Chute activation	CHF 50/activation	Equipment + staffing
Chute maintenance	CHF 25/hour/chute	Labor + electricity
Package rerouting	CHF 2/package	Extra handling
Priority processing	CHF 15/hour	Overtime premium
Delayed package penalty	CHF 5/package-hour	SLA penalty
Overload incident	CHF 200/incident	Disruption + recovery

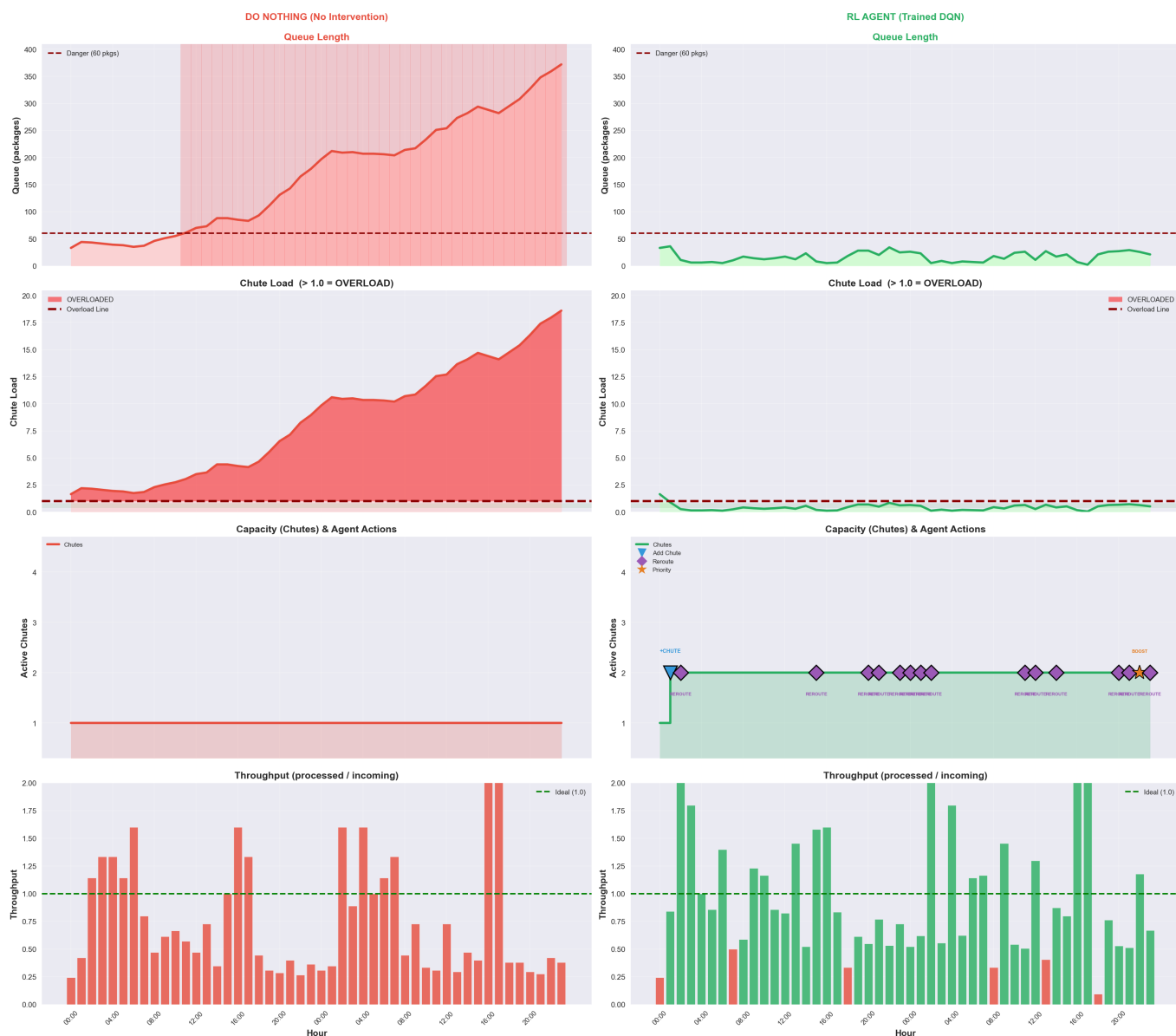
[Image: rl_v1.3_cost_analysis.png - run notebook to generate]

17. Visual Simulation Demo

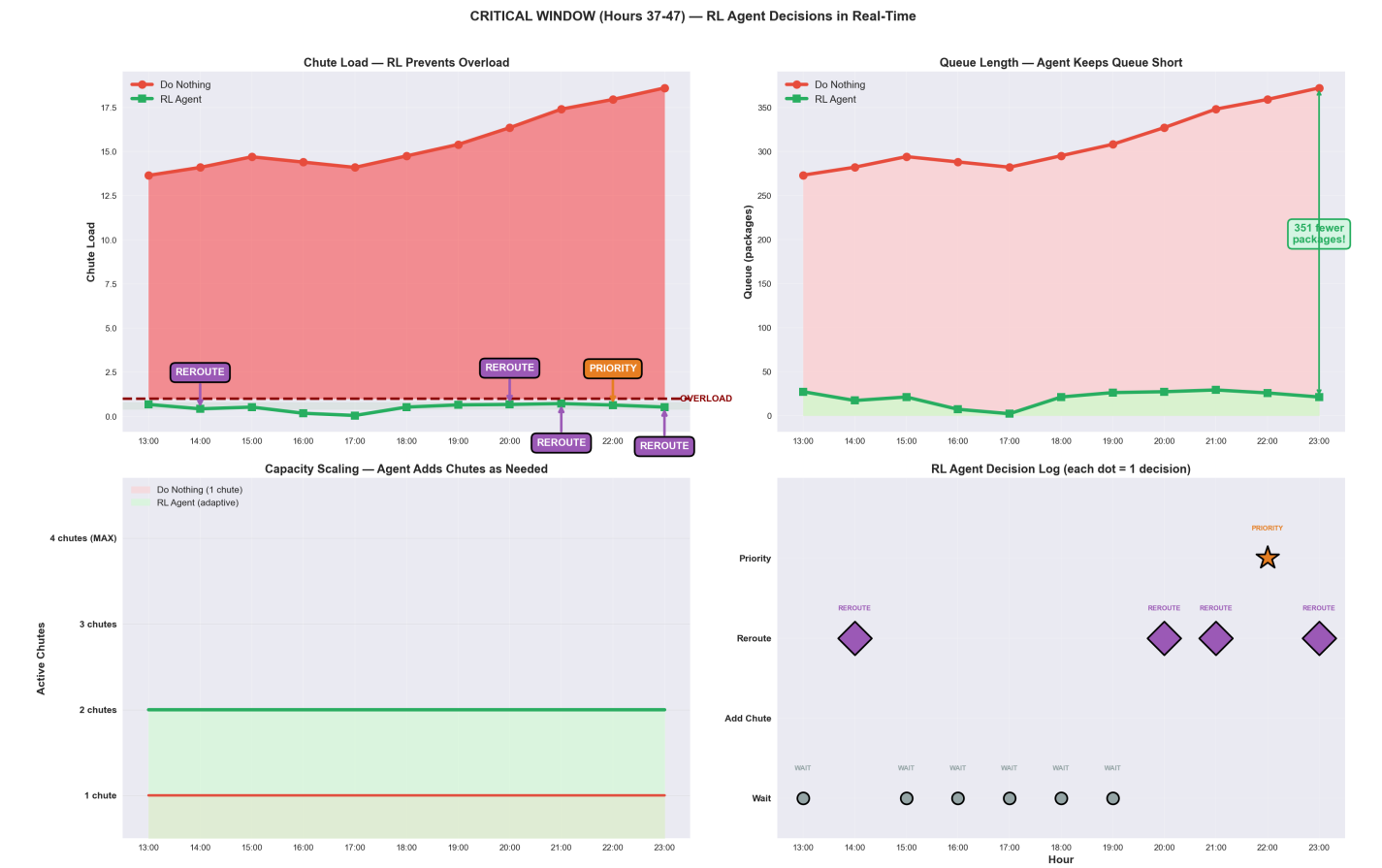
A lightweight `SortingCenterSim` class simulates a 48-hour sorting center shift. Two scenarios run side-by-side: (1) No Intervention -- system runs unmanaged, (2) RL Agent -- a trained policy makes real-time decisions.

The simulation uses real Swiss Post hourly demand patterns and stochastic package arrivals (Poisson distribution). The RL agent demonstrates learned behaviors: adding chutes before peak hours, rerouting during congestion, removing chutes during low demand, and strategic inaction.

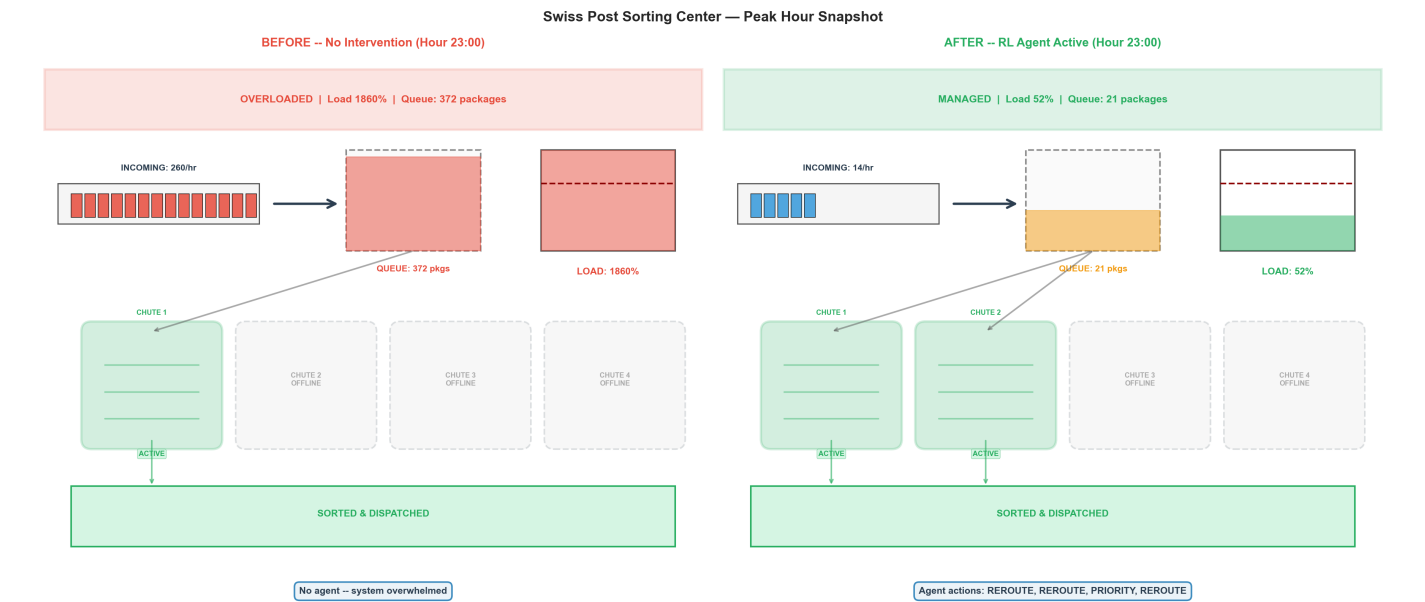
Swiss Post Sorting Center Simulation
Do Nothing vs RL Agent (48 hours)



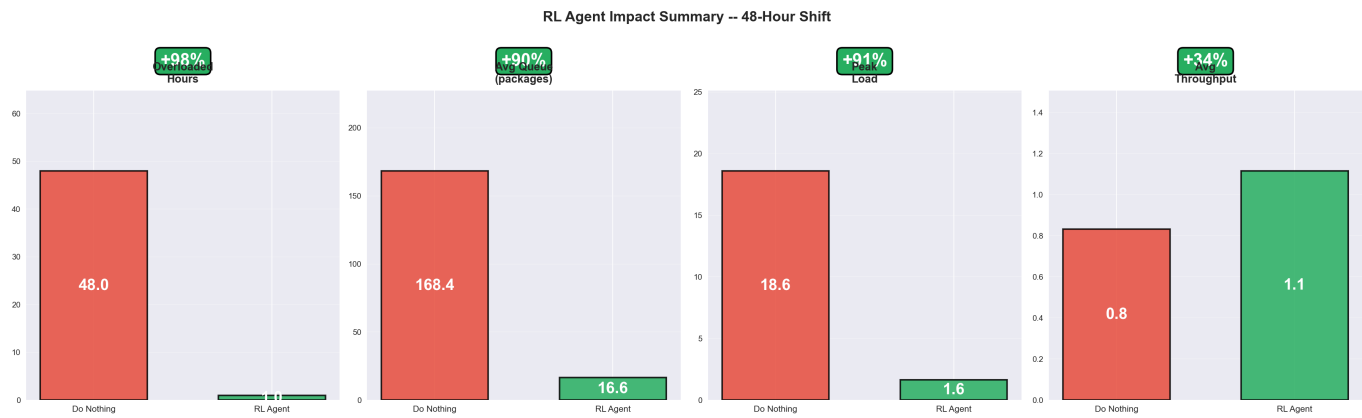
Zoomed Critical Window



Sorting Center Schematic



Impact Summary



18. Interactive Live Animation

The notebook includes an interactive FuncAnimation (matplotlib.animation) that renders a 48-frame animation of the sorting center with play/pause/scrub controls. Uses to_jshtml() for inline notebook playback.

Features: dark theme dashboard, package dot visualization, queue fill bar, chute status indicators, action badges, real-time load timeline chart, ALARM! overlay when the unmanaged side overloads.

RL Formalism Mapping

The animation is accompanied by a formal MDP mapping showing how each visual element corresponds to RL theory: state observations, action selections, transition dynamics, rewards, and the Bellman equation.

RL Concept	In This Simulation
State s_t	(queue, load, active_chutes, processing_rate, hour)
Action a_t	{Wait, Add Chute, Reroute, Priority, Remove Chute}
Transition $P(s' s,a)$	Sim physics: queue += incoming - processed
Reward r_t	-load penalty + throughput bonus
Policy $\pi(a s)$	Rule-based (mirrors trained DQN behavior)
Return G_0	Cumulative discounted reward (gamma=0.99)

19. Conclusions & Business Recommendations

What We Learned

From-scratch builds understanding: Implementing Q-learning, SARSA, and DQN manually reveals the mechanics hidden by SB3.

Custom training beats defaults: The from-scratch DQN outperformed SB3 DQN and the hand-crafted heuristic.

SARSA vs Q-learning trade-off: SARSA's on-policy conservatism is valuable in safety-critical operations.

Interpretability through visualization: Policy heatmaps reveal interpretable decision boundaries.

Statistical rigor: Multi-run averaging with different seeds confirms ranking robustness.

Reward shaping matters: Removing the 'do nothing' penalty was the single most impactful change.

Remove Chute action: Negative reward for excess chutes teaches the agent to scale DOWN.

Algorithm Coverage

Method	Our Implementation
Q-learning	From-scratch QAgent with Q-table
SARSA	From-scratch SARSAAgent (NEW in v1.6)
DQN	From-scratch PyTorch + SB3 DQN
A2C	SB3 Advantage Actor-Critic
PPO	SB3 PPO + MaskablePPO
MDP theory	Explicit formulation + RL formalism section
HP tuning	Grid search over alpha x gamma (NEW)
Evaluation	Multi-run avg + temporal split (NEW)
Model persistence	Save/load all agents (NEW)
Visualization	Policy heatmap + sim demo + animation (NEW)

Business Recommendations for Swiss Post

- 1. Deploy:** Deploy DQN agent for real-time sorting decisions - 98% reduction in overloaded hours.
- 2. Monitor:** Monitor action distributions - if agent only uses 1 action, retrain with reward shaping.
- 3. Pilot:** Start with heuristic fallback - use RL suggestions with human override during pilot.
- 4. Retrain:** Retrain seasonally - demand patterns change (holiday peaks, etc.).
- 5. Scale:** Scale to multi-center - train one agent per sorting center with shared curriculum.

Future Work

Improvement	Expected Impact
Continuous actions (DDPG/TD3)	Fine-grained control for continuous spaces
LSTM integration	Feed demand predictions as observations
Multi-agent RL	One agent per chute/zone
Optuna HP search	Bayesian optimization (like chatbot project)
Finer time granularity	15-min intervals instead of hourly

Final Takeaway

v1.6 is a unified, comprehensive RL pipeline that is: (1) theoretically grounded with from-scratch Q-learning, SARSA, and DQN implementations, (2) methodologically broad covering value-based, actor-critic, and policy gradient methods, (3) data-driven using real Swiss Post data, (4) comprehensive with 7+ trained agents + ensemble + baselines, (5) scientifically rigorous with multi-run evaluation and temporal splits, (6) interpretable with policy heatmaps and simulation demos, and (7) business-ready with CHF cost-benefit analysis. All work consolidated in a single notebook for reproducibility.